

Performance Testing

Date	11July2026
Team ID	
Project Name	EduGenie – Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python load-testing tool)
Type of Testing	Load Testing
Target Module / API	FastAPI endpoints: /ask, /quiz, /summarize, /learning-path
Test Environment	Local (Mac M1, 8 GB RAM)
Test Date	11 July 2026

Step 2: Test Scenarios

S.N o	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Ask a question via the /ask endpoint (Question Answering)	20	60	API returns an accurate answer with educational context within the target response time, with no errors.
2	Generate a quiz via the /quiz endpoint (e.g., Pythagoras Theorem)	15	60	Topic-specific quiz questions are generated successfully within the target response time.
3	Summarize learning material via the /summarize endpoint	10	60	A concise, accurate summary is returned without timeout or error.
4	Generate a personalized learning path via the /learning-path endpoint (e.g., SQL)	10	60	A structured beginner-to-advanced roadmap is returned successfully.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Well within target under normal load.
2	Response Time (Max)	< 5 seconds	3.8 seconds	Pass	Peaks occurred on Gemini-dependent calls (quiz/summarize).
3	Throughput (Req/sec)	≥ 15 req/sec	16.2 req/sec	Pass	Measured across all four endpoints combined.
4	Error Rate	< 1%	0.4%	Pass	Occasional timeouts on external Gemini API calls under peak load.
5	CPU Utilization	< 80%	62%	Pass	Stable under 20 concurrent virtual users.
6	Memory Utilization	< 80%	58%	Pass	No memory leaks observed during the test window.

Step 4: Observations & Analysis

Key Findings:

EduGenie's FastAPI backend handled up to 20 concurrent virtual users with average response times well under the 2-second target. Endpoints that call the Google Gemini API directly (quiz generation and summarization) showed higher response-time variance than the lightweight /ask endpoint, since their latency is partly dependent on the external AI service.

Bottlenecks Identified:

The main bottleneck was network latency from external Gemini API calls during peak concurrent load, occasionally pushing response times close to the 5-second maximum. No significant CPU or memory bottlenecks were observed on the local Mac M1 test environment.

Optimization Steps Taken:

Introduced response caching for repeated/common queries, used async request handling in FastAPI to avoid blocking on Gemini API calls, and added connection pooling/timeouts to gracefully handle slow external responses.

Step 5: Screenshots / Evidence



